

Software Development Fundamentals (SDF) – I

ODD 2023



Control flow in C

**Jaypee Institute of Information Technology (JIIT),
Noida**

Outline

- Control Flow in C
- Loops in C
- Conditional statement in C
- Switch Case in C

- In program there are multiple statements. In general, statements are executed in sequentially in the order in which they appear in the program.
- But if we want that a particular statement executes only if condition satisfied or some statements executes several times.
- In that case we need to define the control flow of the statements.
- Control statements enable us to specify the order in which the various instructions in the program are to be executed.
- This determines the flow of control.
- Control statements define how the control is transferred to other parts of the program.

- C language supports four types of control- statements, which are as-

1. if...else

2. goto

3. switch

4. loop

- while

- do...while

- for

If statement

- If statement are decision control statement.
- Condition is written inside the round bracket written just after if.
- **If** the condition is true then a statement or a block of statements is executed (one part of the program) otherwise statements inside “if “ will not be executed.
- After execution of **if** statement, rest statements of the program will be executed.

Syntax:

```
if(condition)
{
    statement;
    .....
}
```

Conditional statements - if

- Example :-

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int num1, num2;
```

```
    scanf("%d%d",&num1,&num2);
```

```
    if(num1 == num2)
```

```
    {
```

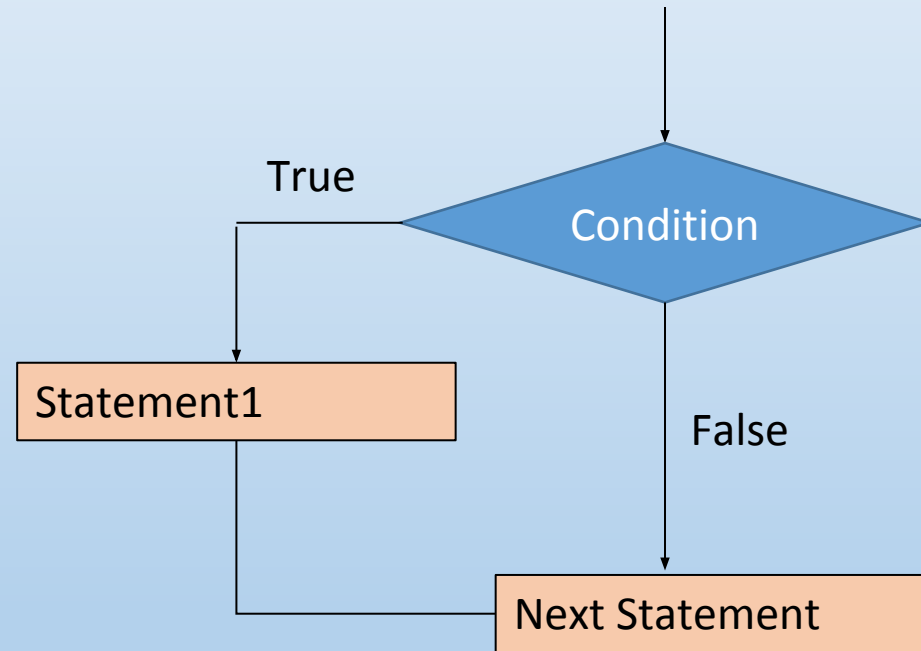
```
        printf(" numbers are equal");
```

```
    }
```

```
    return 0;
```

```
}
```

Control Flow of If statement



If-Else Statement

- In the If-else statement if condition inside if is true then block of statements inside if will be executed otherwise block of statements inside else will be executed.
- After execution of **if –else** statement, rest statements of the program will be executed.

```
if(condition)
```

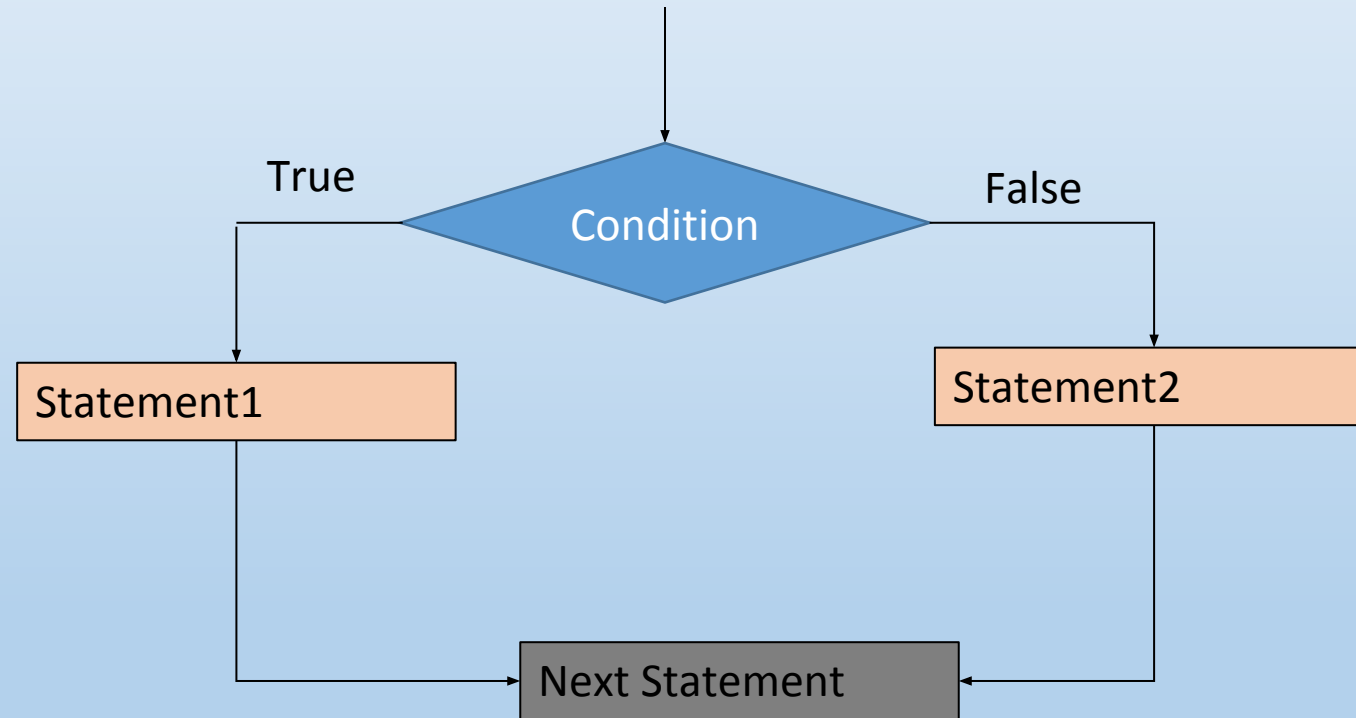
```
{  
    statement1;  
    .....  
}
```

```
else
```

```
{  
    statement2;  
    .....  
}
```

Here if the condition is true then statement1 is executed and if it is false then statement2 is executed.

Control Flow of If-else statement



- **Example**

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int num1, num2;
```

```
    scanf("%d%d",&num1,&num2);
```

```
    if(num1 == num2)
```

```
    {
```

```
        printf(" numbers are equal");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("numbers are not equal");
```

```
    }
```

```
    return 0;
```

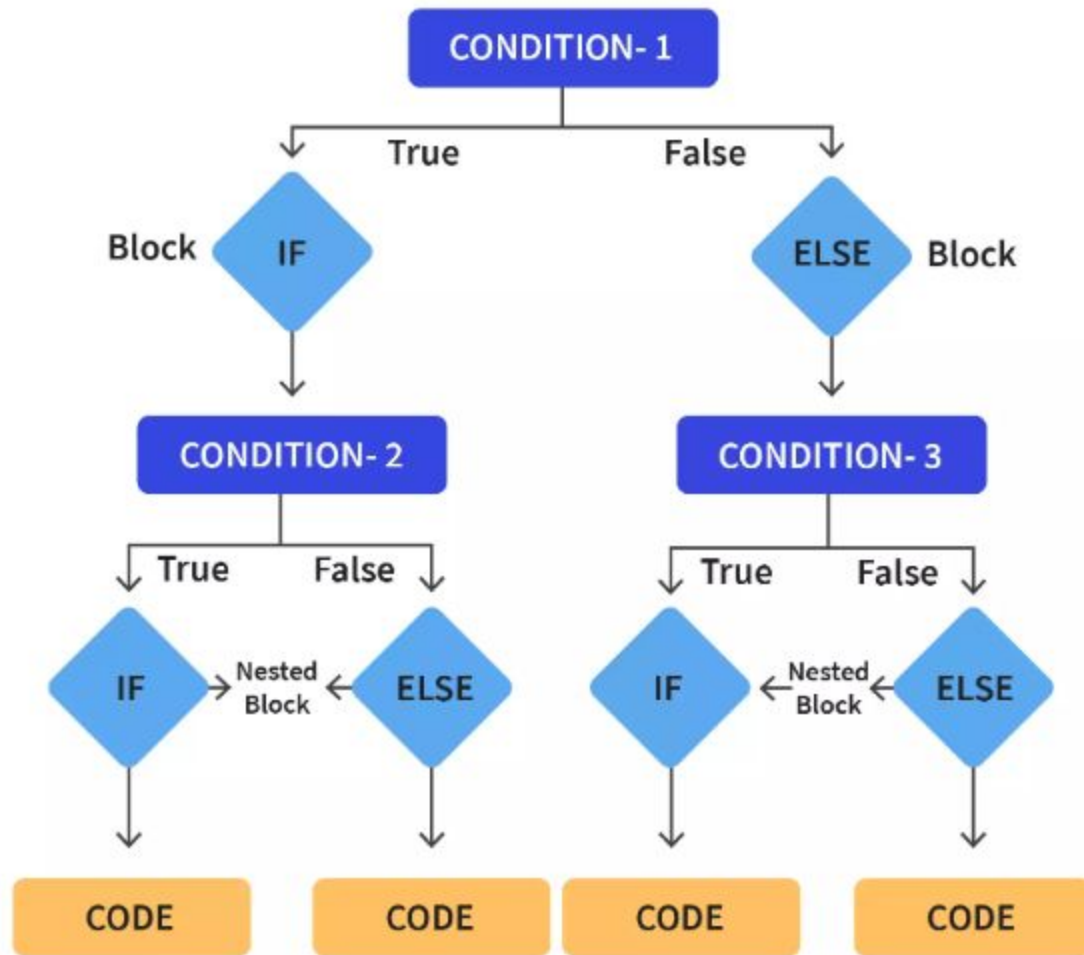
```
}
```

Nesting of if... else

- We can have multiple if-else inside if or else block.
- This is called nesting of if... else statements.

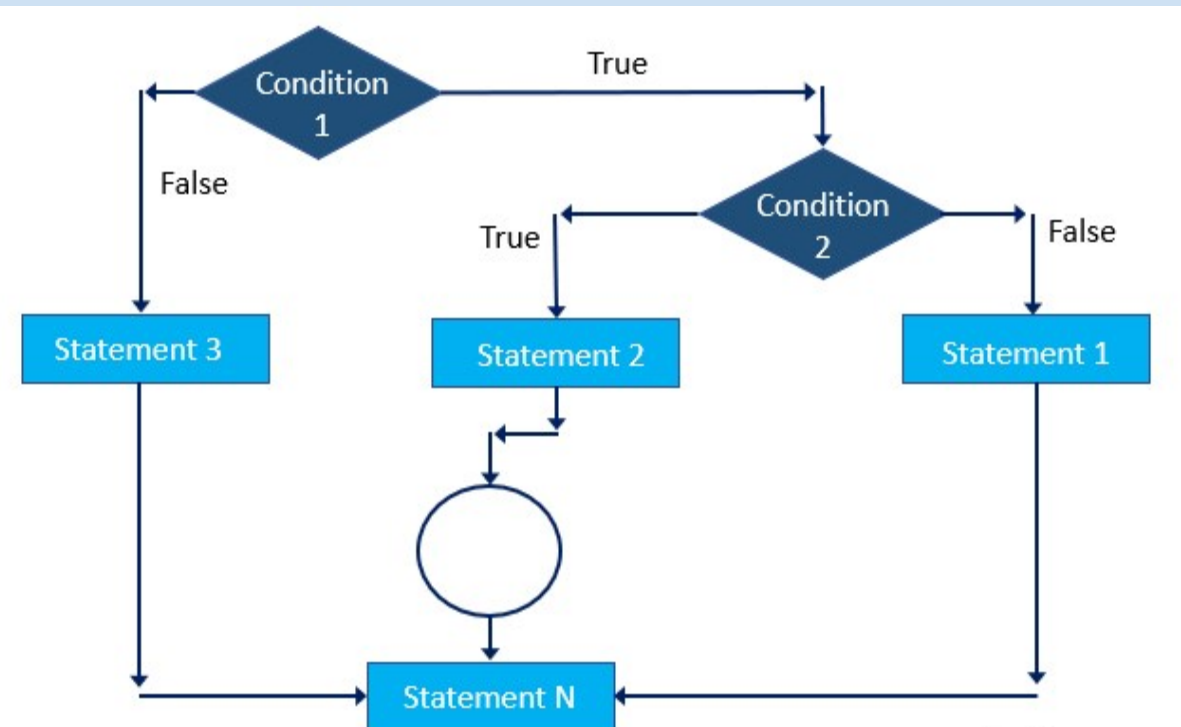
Syntax:

```
if (condition 1)
{
    if (condition 2)
        statementA1;
    else
        statementA2;
}
else
{
    if(condition 3)
        statementB1;
    else
        statementB2
}
```

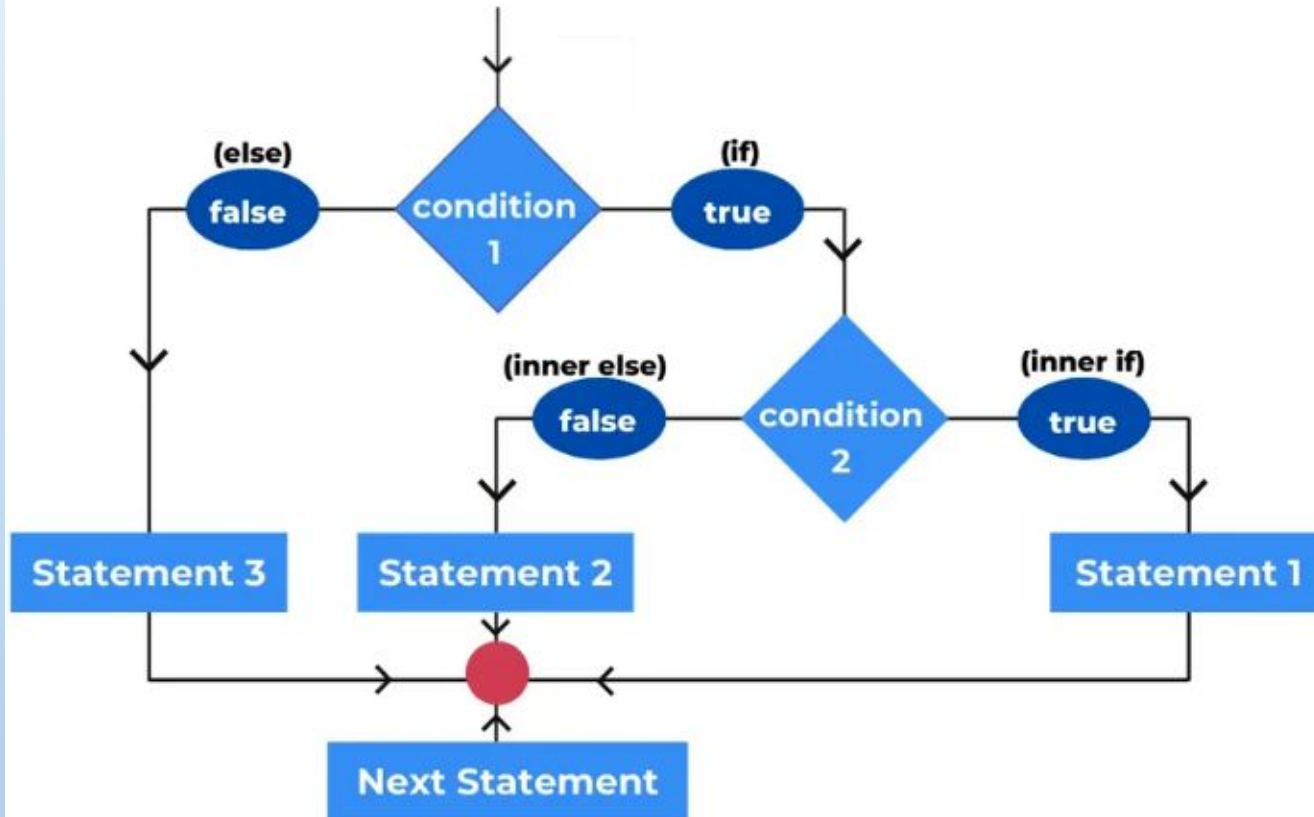


Example-1

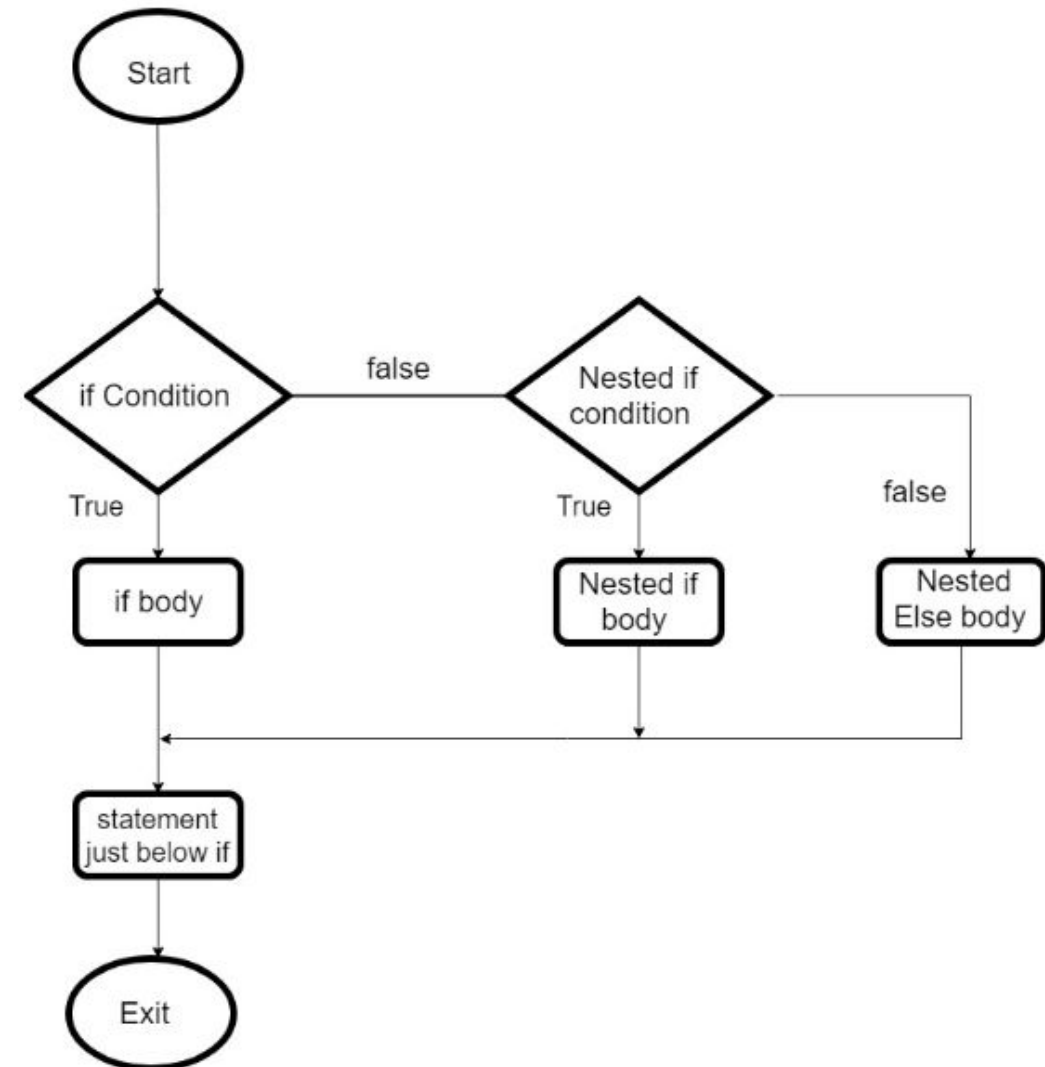
Example-2



Nested If Statements



Example-3



Example-4

Program to find student highest marks out of three subject

```
main( )
{
    int marks1, marks, marks3, highest;
    printf ("Enter the marks of three subjects") ;
    scanf ("%d%d%d",&marks1,&marks2,&marks3) ;
    if (marks1> marks2)
    {
        if(marks1 > marks3)
            highest= marks1;
        else
            highest= marks3;
    }
    else
    {
        if (marks2 > marks3)
```

```
        highest= marks2;
    else
        highest= marks3;
    }
    printf ("highest marks %d\n",
highest) ;
}
```

Output:

Enter the marks of three subjects

30

40

50

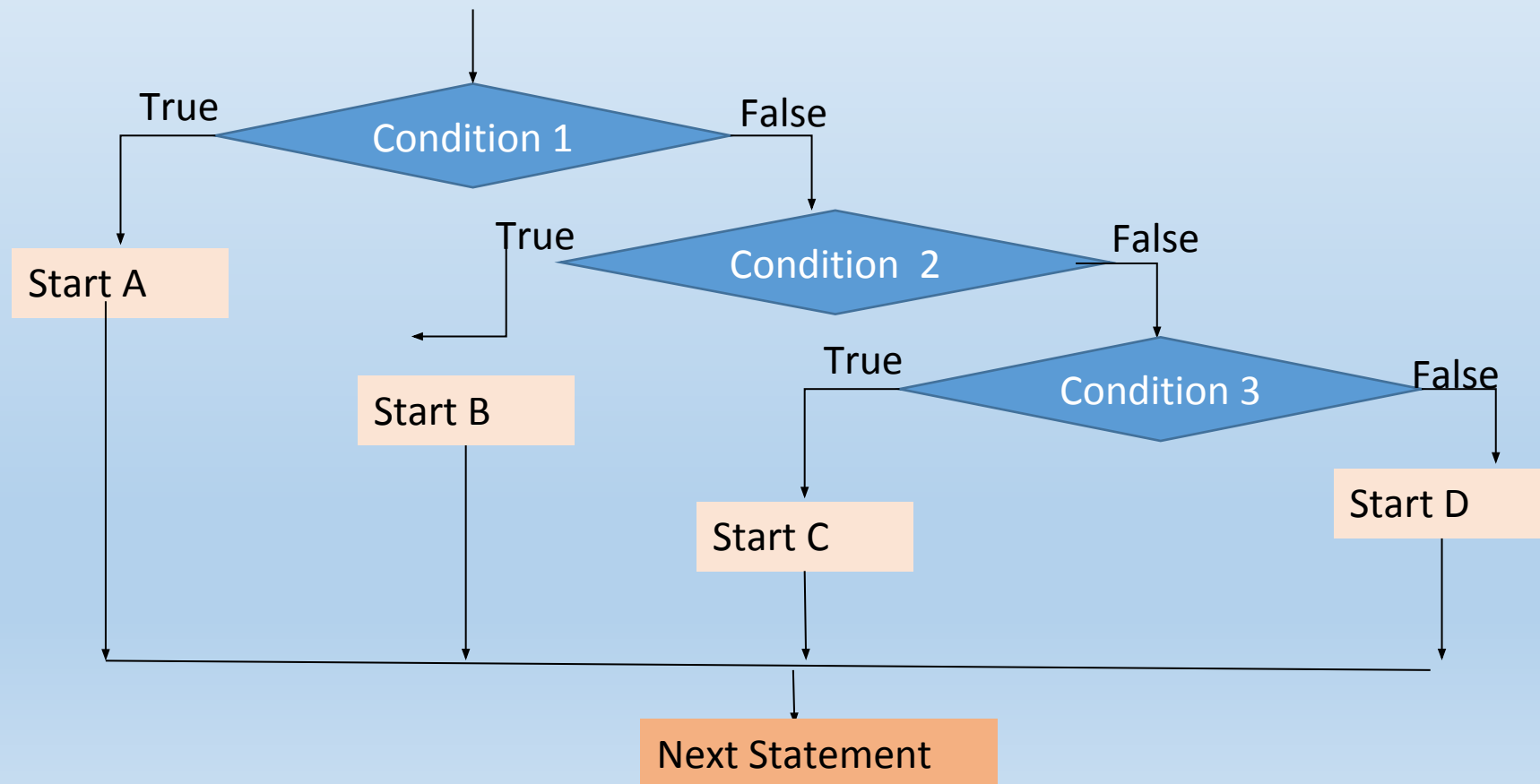
Highest marks 50

else if Ladder

- This is a type of nesting in which there is an if-else statement in every else part except the last else part.
- This type of nesting is frequently used in programs and is also known as else if ladder.

```
if ( condition!)  
    statementA;  
else if (condition2)  
    statementB;  
else if ( condition 3)  
    statementC;  
else  
    statementD;
```

Here each condition is checked, and when a condition is found to be true, the statements corresponding to that are executed, and the control comes out of the nested structure without checking remaining conditions. If none of the conditions is true then the last else part is executed.



WAP to find out the grade of a student when the marks of 4 subjects are given

```
#include<stdio.h>

main ( )
{
float m1, m2, m3, m4, total, per;
char grade;
Printf("Enter the marks of four subjects ");
scanf("%f%f%f%f",&m1,&m2,&m3,&m4) ;
total=m1+m2+m3+m4;
per=total/4;
if (per>=85)
    grade= 'A' ;
else if(per>=70)
    grade='B';
```

```
else if (per>=55)
    grade= 'C' ;
else if (per>=40)
    grade= 'D' ;
else
    grade='E';
printf ("Percentage is %f\nGrade is %c\n", per, grade) ;
```

Enter the marks of four subjects

80

70

60

90

Percentage is 75 Grade is B

Practice questions

- Check if a given number is positive / negative / zero
- Check whether a given number is positive and greater than 10 or negative or zero
- Given a year, find if it is a leap year or not.
- Temperature conversion – if the temperature is given in Celsius, convert to Fahrenheit or vice-versa.
- Check if a given number is even or odd.
- Check whether the triangle is equilateral or scalene or isosceles.
- Check if a character is an alphabet or not.
- Check if a character is a vowel or consonant.
- Print the number of days in a given month

Practice questions

- Given the angles of a triangle, check whether the given triangle is valid or not.
- Given the sides of a triangle, check whether the given triangle is valid or not.
- Given the marks of a student, find his/her grade.

Switch case

Switch case in C

- For multiple conditioned statements, where only one option is selected from the set of options which is commonly known as menu- driven program.
- C provides the 'switch' multiple selection structure to decide which statement to execute next.

Syntax:

```
switch(expression)
{
    case constant1:
        statement
    case constant2:
        statement
    case constantN:
        statement
    default
        statement }
```

- Here **switch**, **case**, and **default** are keywords.
- The switch statement evaluates an expression, then attempts to match the result to one of several possible cases
- The match must be an exact match.
- The number of 'case' within the 'switch' statement depends upon the number of options present in menu.
- The switch is the "controlling expression"
 - Can only be used with constant integer expressions.
 - Since, a single character is a small positive integer. It is allowed to use as controlling expression.
 - The expression appears in ()

- The case is a "label"
 - The label must be followed by a " : "
 - Braces, { }, not required around statements
- Default is an “optional” section
 - It is generally used for error handling such as wrong choice entry or out of range choice.
 - The default section must be followed by a “:”
 - It is automatically executed when no exact label match is found.
 - When the label in a case matches the controlling value, all of the statements in the rest of the switch structure take place. (this does not happen with if-else structure)

Valid and Invalid Switch Cases

Some valid and invalid ways of writing switch expressions and case constants.

```
int a, b, c;    char d, e;    float f;
```

Valid:

```
switch(a)  switch(a>b)  switch(d+e-3)  switch(a>b&&b>c)
switch(func(a, b))
```

Invalid

```
switch(f)    switch(a+4.5)
```

Valid

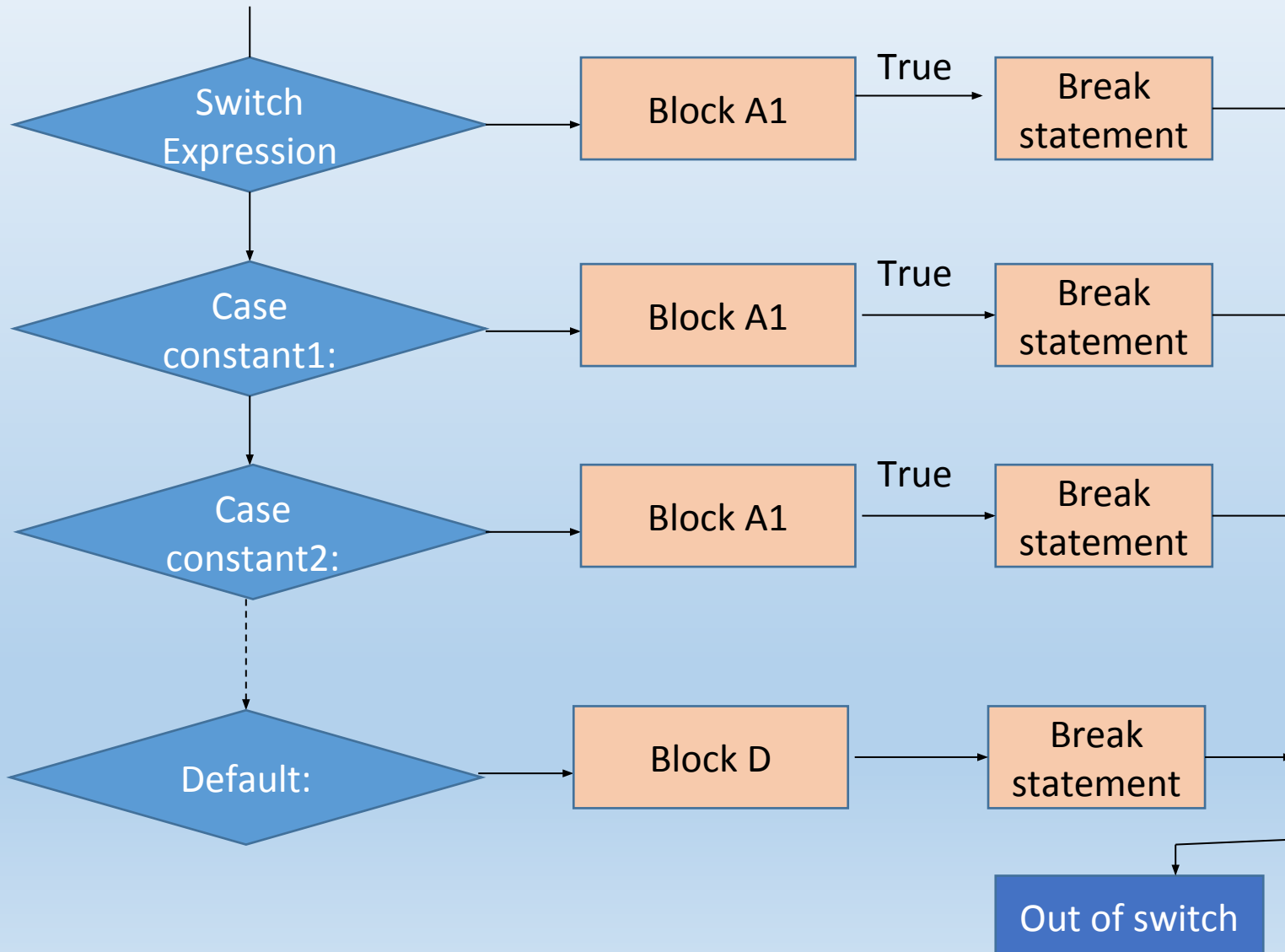
```
case 4:    case 'a':    case 2+4:    case 'a'>'b':
```

Invalid

```
case "second":    case 2.3:    case a:    case a>b:    case a+2:    case 2, 4, 5:
case 2 : 4 : 5 :
```

- Firstly the switch expression is evaluated, then the value of this expression is compared one by one with every case constant.
- If the value of expression matches with any case constant, then all statements under that particular case are executed.
- If none of the case constant matches with the value of the expression then the block of statements under default is executed.
- 'default' is optional, if it is not present and no case matches then no action takes place.
- These cases and default can occur in any order.

Control Flow of switch case:



Program to understand the switch control statement*/

```
#include<stdio.h>
void main ()
{
int ch;
printf ("Enter your choice to print 1.Hi, 2.Bye, 3.Welcome ") ;
scanf("%d",ch);
switch(ch)
{
case 1:
printf("Hi\n") ;
case 2:
printf("Bye\n");
case 3:
printf("Welcome\n");
default:
printf ("Wrong choice\n");
}
```

Output:

Enter your choice to print 1.Hi, 2.Bye, 3.Welcome
3
Welcome

Switch Case □ break

- The break statement can be used as the last statement in each case's statement list
- A break statement stops from executing all of the remaining statements in the rest of the switch structure after the label in a case matches the controlling value and jump to the end of the switch statement.
- If a break statement is not used, the flow of control will continue into the next case

Switch Case with break

Syntax :-

```
switch (controlling expression/ Variable)
{
    case case_1_label :
        statements ;
        break;
    case case_2_label :
        statements ;
        break;
    default :
        statements ;
}
```

Switch Case □ An example to Illustrate Switch-Case

WAP to accept day no. (1 to 7) and print the day name

```
#include<stdio.h>
void main()
{
    int num;
    printf("enter day number");
    scanf("%d",&num);
    switch(num)
    {
        case 1:
            printf("monday");
            break;
        case 2:
            printf("tuesday");
            break;
        case 3:
            printf("wednesday");
            break;
```

```
        case 4:
            printf("thursday");
            break;
        case 5:
            printf("friday");
            break;
        case 6:
            printf("saturday");
            break;
        case 7:
            printf("sunday");
            break;
        default:
            printf("Invalid input");
    }
}
```

GOTO Label

- By default, a program executes the statements from top to bottom.
- GOTO statement is used to move the control from one position to another desired position within a program.

Syntax:-

```
label_name:  
(statements);  
goto label_name;  
(statements);  
}
```

- Label name is any user defined name or identifiers (similar to the variable name)
- label is followed by colon (:) sign at the time of declaration and followed by semicolon (;) at the time of transfer calling.
- Label must be indicated at that position in the program where the control is need to be transfer.

GOTO label

- Example :-

```
#include<stdio.h>
int main()
{
    int age =19;
    g: //label name
    printf("you are Eligible\n");
    s: //label name
    printf("you are not Eligible\n");
    age--;
    if(age>18)
        goto g; //goto label g
    if (age>=17)
        goto s; //goto label s
    return 0;
}
```

- Output :-

```
you are Eligible
you are not Eligible
you are not Eligible
you are not Eligible
```

Remember!!

- **The Switch expression should result in a constant value**
- **Expression value should be of datatype - int or char only.**
- **Case Values must be Unique**
- **Nesting of switch Statements is allowed.**

Practice questions

- Write a C program print the total number of days in a month using switch case.
- Write a C program to check whether an alphabet is a vowel or consonant using switch case.
- Write a C program to find the roots of a quadratic equation using switch case.
- Write a C program to create Simple Calculator using switch case.
- Compare between Switch-Case and If-Else-if ladder.

LOOPS in C

- Loops are used when we want to execute a part of the program or a block of statements several times.
- For example, suppose you want to print “welcome to C” 50 times. One way to print 50 times is write 50 printf statements, which is not an efficient practice.
- Using loop you can write one loop statement and only one printf statement, and this approach is definitely better than the first one.
- With the help of loop we can execute a part of the program repeatedly till some condition is true.

There are three loop statement in C:

- while
- do while
- for

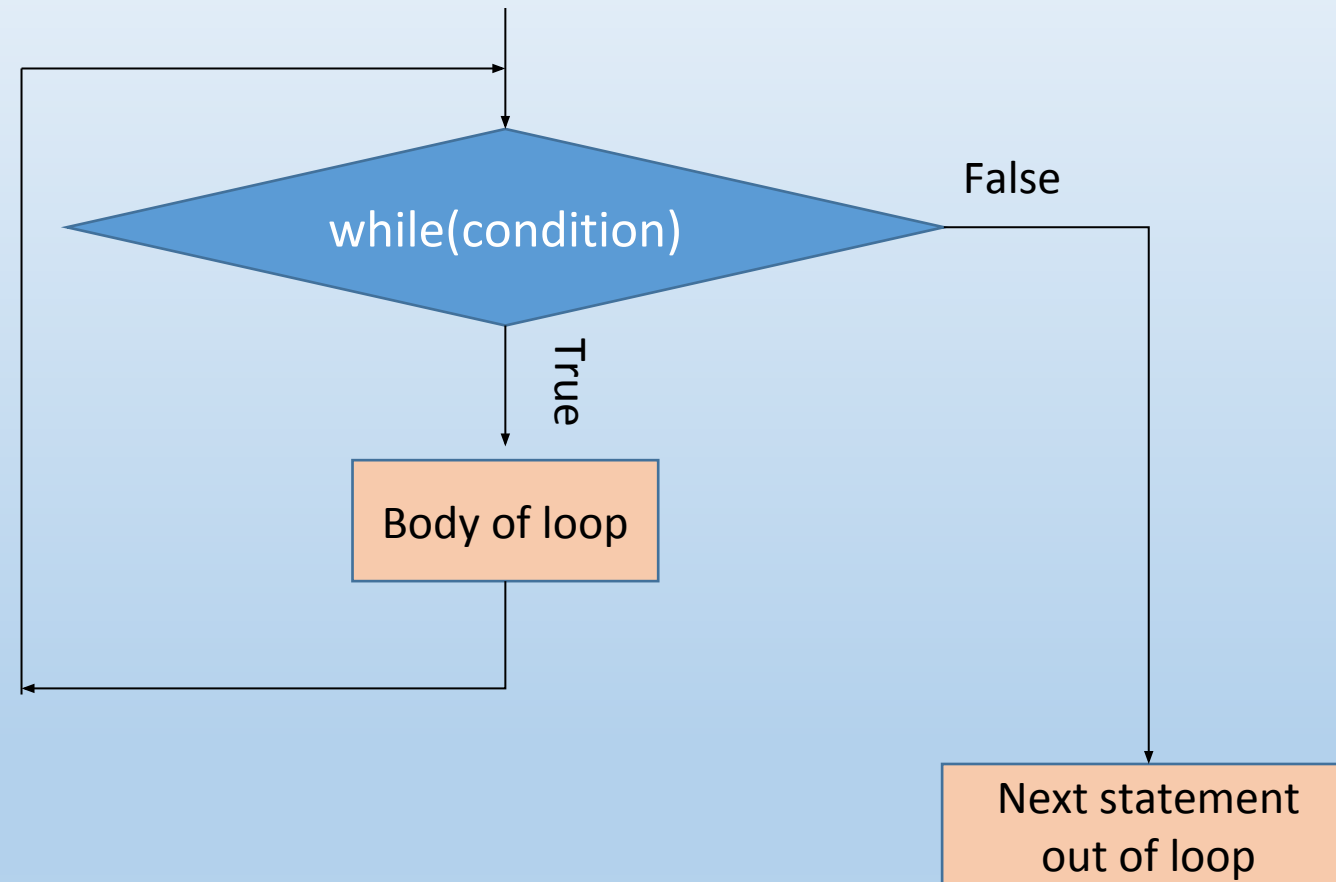
While Loop

While statement can be written as:

```
while(condition)
{
    statement
    .....
}
```

- First the condition is evaluated, if it is true then the statements in the body of loop are executed.
- After the execution, again the condition is checked and if it is found to be true then again the statement in the body of loop are executed.
- This means that these statements are executed continuously till the condition is true and when it becomes false, the loop terminates and the control comes out of the loop.
- Each execution of the loop body is known as iteration.

Control flow of while loop



Program to print “welcome to C” 50 times using while loop

```
#include<stdio.h>

main ( )
{
    int i=1;
    while(i<=50)
    {
        printf("\n Welcome to C") ;
        i=i+1; / * Statement that changes the value of condition * /
    }
}
```

Output:

Welcome to C
Welcome to C
Welcome to C

.
. .
. .

50 times

Do-while loop

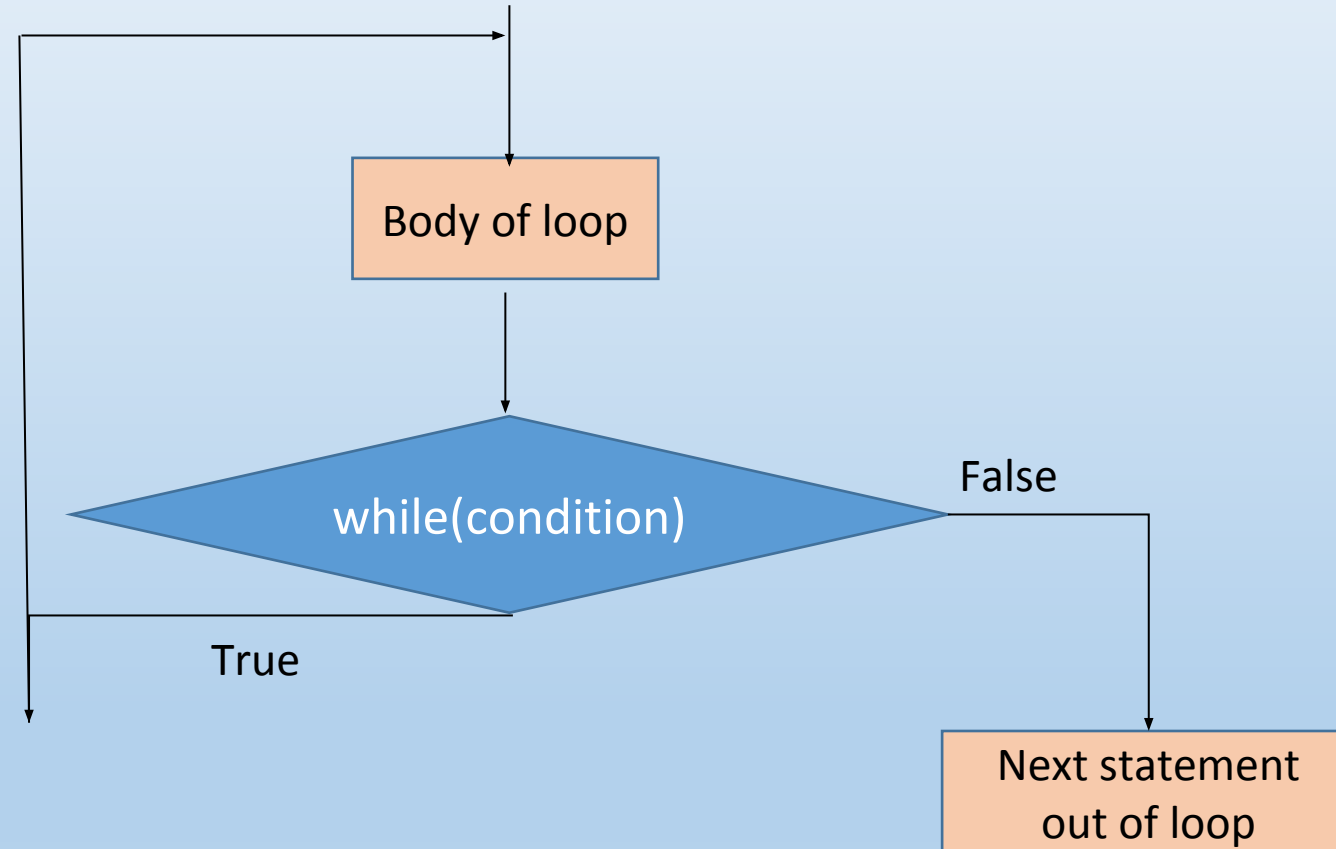
- 'do...while' statement is also used for looping.

The syntax for writing this loop is:

```
do
{
    statement
    statement
} while(condition);
```

- Here firstly the statements inside loop body are executed and then the condition is evaluated.
- If the condition is true, then again the loop body is executed and this process continues until the condition becomes false.
- Note that unlike while loop, here a semicolon is placed after the condition.

Control flow of do-while loop



Program to print “welcome to C” 50 times using do-while loop

```
#include<stdio.h>
main ( )
{
    int i=1;
do
{
    printf("\n Welcome to C") ;
    i=i+1; / * Statement that changes the value of condition
} while(i<=50);

}
```

Output:

Welcome to C
Welcome to C
Welcome to C

·
·
·
·

50 times

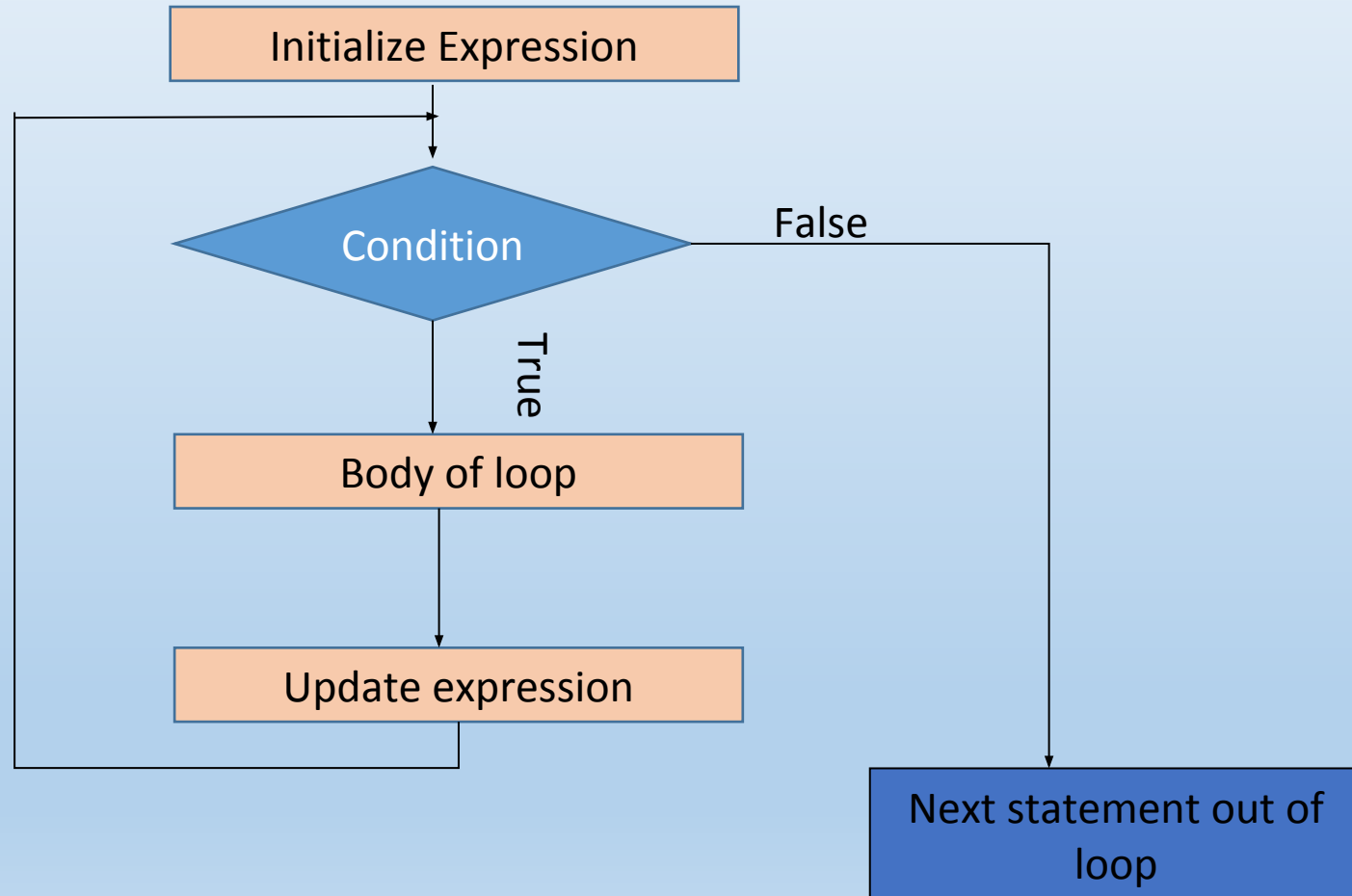
for loop

- The 'for' statement is very useful while programming in C.
- It has three expressions and semicolons are used for separating these expressions.
- The 'for' statement can be written as-

```
for(expression1;expression2;expression3)
    statement;
for(expression1;expression2;expression3)
{
    statement;
    statement;
}
```

- Expression 1 is an initialization expression, expression2 is a test expression or condition and expression3 is an update expression.
- expression1 is executed only once when the loop starts and is used to initialize the loop variables.
- This expression is generally an assignment expression.
- expression2 is a condition and is tested before each iteration of the loop.
- This condition generally uses relational and logical operators.
- expression3 is an update expression and is executed each time after the body of the loop is executed.
- This process continues till the condition is true and when the condition becomes false.
- Then the loop is terminated and control is transferred to the statement following the loop.

Control flow of for loop



Program to print “welcome to C” 50 times using while loop

```
#include<stdio.h>
main ( )
{
    int i=1;
    for(i=1;i<=50;i++)
    {
        printf(“\n Welcome to C”) ;
    }
}
```

Output:
Welcome to C
Welcome to C
Welcome to C
.
.
.
.
50 times

Practice

- Write a program in C language for printing all the odd / even numbers within a given range using the while loop.
- Write a program in C language to print the addition of numbers within a range.
- Write a program in C language to print the factorial of a number.
- Write a program in C language to print all the uppercase letters.
- Write a program in C language to print the multiplication table of an integer.
- Write a program to print the square roots of all the numbers within a range.
- Write a program in C to print a range of numbers (from a to b) that are divisible by 2, 3, and 5.
- Write a program in C to print arithmetic progression and its sum.
- Write a program in C to print right angled triangle star pattern.

The End